

LeetCode 697

Degree of an Array — Shortest Subarray

| Algorithm Notes

- **Basic Idea:** Store the frequency of each number and find the degree of the array.
- **Tracking Positions:** Also store the first and last occurrence of every number during traversal.
- **Calculating Answer:** Once the frequency map is built, for any number whose frequency is equal to the degree, note its length ($\text{last} - \text{first} + 1$) and find the minimum possible length to get the shortest subarray.

| Source Code (C++)

```

class Solution {
public:
    int findShortestSubArray(vector<int>& nums) {

        //basic idea ill use is
        //store the frequency of each number and find degree
        //also store first and last occurrence of every number

        //once freq map is built
        //any number whose freq is equal to degree
        //we will note its last-first and see if its less than ans

        unordered_map<int,int> freq;
        unordered_map<int,int> first;
        unordered_map<int,int> last;
        int n=nums.size();
        int deg=1;
        for(int i=0;i<n;i++){

            freq[nums[i]]++;
            deg=max(deg,freq[nums[i]]);

            if(first.find(nums[i])==first.end()){
                first[nums[i]]=i;
            }

            last[nums[i]]=i;
        }

        int ans=INT_MAX;
        for(auto &it:freq){
            int num=it.first;
            int frequ=it.second;
            if(frequ==deg){
                ans=min(ans,last[num]-first[num]+1);
            }
        }

        return ans;
    }
};

```